

COMPLETE STUDY NOTES

DevOps

Lifecycle · Git · Jenkins · Docker · Kubernetes · Prometheus

Six units · 17 diagrams · worked examples · command cheat sheet

Prepared for classroom revision and fast recall

Unit 1 Introduction · Unit 2 DevOps & Lifecycle · Unit 3 Version Control
Unit 4 Continuous Integration & Jenkins · Unit 5 Deployment & Containers
Unit 6 Kubernetes & Monitoring · Appendix Rapid Revision

Contents

Unit 1 — Introduction: Objective, Scope and Outcome

- 1.1 Why this course exists
- 1.2 Objective of the course
- 1.3 Scope
- 1.4 Outcome

Unit 2 — Introduction to DevOps & Lifecycle

- 2.1 What DevOps is
- 2.2 Goals of DevOps
- 2.3 Benefits of DevOps
- 2.4 The DevOps lifecycle
- 2.5 Roles and responsibilities of a DevOps engineer
- 2.6 The four DevOps principles
- 2.7 Agile vs DevOps
- 2.8 Infrastructure as Code (IaC)

Unit 3 — Version Control Systems

- 3.1 What a Version Control System is
- 3.2 Types of Version Control Systems
- 3.3 Git fundamentals
- 3.4 Git repository concepts — the four areas
- 3.5 Essential Git commands
- 3.6 Branching and merging
- 3.7 Repository management

Unit 4 — Continuous Integration & Jenkins

- 4.1 What Continuous Integration is
- 4.2 CI workflow
- 4.3 CI tools overview
- 4.4 Introduction to Jenkins
- 4.5 Jenkins architecture (Controller-Agent)
- 4.6 Jenkins dashboard overview
- 4.7 Jenkins jobs — Freestyle vs Pipeline
- 4.8 Jenkinsfile basics
- 4.9 Build automation concepts
- 4.10 Automated testing in the CI pipeline
- 4.11 Continuous feedback in the CI process

Unit 5 — Continuous Deployment & Containerization

- 5.1 Continuous Delivery vs Continuous Deployment
- 5.2 The CI/CD pipeline as a whole
- 5.3 Deployment strategies
- 5.4 Configuration management
- 5.5 Virtualization vs Containerization
- 5.6 Docker basics
- 5.7 Docker architecture
- 5.8 Docker commands

Unit 6 — Kubernetes Fundamentals & Monitoring

- 6.1 Introduction to Kubernetes

- 6.2 Kubernetes architecture
- 6.3 Pods and containers
- 6.4 Container orchestration basics
- 6.5 Monitoring and observability
- 6.6 Prometheus
- 6.7 Time series data
- 6.8 Metrics collection and metric types
- 6.9 Prometheus architecture
- 6.10 Grafana and dashboard creation
- 6.11 Logging and alerting
- 6.12 Performance monitoring basics

Appendix — Rapid Revision Sheet

- A. One-line definitions
- B. Command cheat sheet
- C. Comparisons most likely to be asked
- D. The whole pipeline in one paragraph

Unit 1 — Introduction: Objective, Scope and Outcome

1.1 Why this course exists

For decades software was built by two teams that barely spoke to each other. Developers were paid to ship new features. Operations was paid to keep the servers stable. Every new feature is a risk to stability — so the two teams had exactly opposite incentives. The result was the famous wall of confusion: developers threw code over the wall, operations caught it, it broke, and each side blamed the other.

DevOps is the answer to that problem. It is not a tool and not a job title — it is a **culture plus a set of practices** that makes one team responsible for a product from the first line of code to the last user request in production.

1.2 Objective of the course

- Understand what DevOps actually is, and why it grew out of Agile.
- Learn the DevOps lifecycle end to end and the tool that sits at each stage.
- Be able to use Git confidently — branch, merge, resolve conflicts, manage a repository.
- Build an automated CI pipeline in Jenkins that compiles, tests and reports on every commit.
- Package an application with Docker and understand how containers differ from virtual machines.
- Deploy and orchestrate containers with Kubernetes.
- Monitor a running system with Prometheus and visualise it with Grafana.

1.3 Scope

The course covers the complete delivery chain. Each unit is one link in that chain:

Unit	Stage of the chain	Core tool
2	Culture, lifecycle, principles, IaC	Concepts + Terraform/Ansible
3	Write and version the code	Git / GitHub
4	Build and test it automatically	Jenkins
5	Package and release it	Docker
6	Run it at scale and watch it	Kubernetes, Prometheus, Grafana

1.4 Outcome — what you will be able to do

By the end you should be able to take a piece of source code and describe, and largely build, the full path it travels: commit → automated build → automated test → container image → deployment to a cluster → dashboards and alerts. That path is exactly what a junior DevOps engineer is hired to maintain.

HOW TO USE THESE NOTES

Each unit follows the same shape: the idea in plain words, a diagram, then a concrete example or command set, then a summary table. For exams, the **tables and diagrams** are the highest-value revision material. The cheat sheet at the end of the document collects every command in one place.

Unit 2 — Introduction to DevOps & Lifecycle

2.1 What DevOps is

Definition: DevOps is a culture and set of practices that unites software Development (Dev) and IT Operations (Ops) so that an organisation can deliver software faster, more frequently and more reliably, through collaboration and automation.

Break the definition into its three real parts:

1. **People / culture** — shared ownership. "You build it, you run it." No blame, no wall.
2. **Process** — small changes released often, with feedback after every change.
3. **Tools** — automation for build, test, deploy and monitoring, so humans stop doing repetitive work.

EXAM TRAP

DevOps is NOT a tool, NOT a single person's job, and NOT just "Jenkins + Docker". Tools support DevOps; they are not DevOps. If a company installs Jenkins but Dev and Ops still blame each other, that company is not doing DevOps.

2.2 Goals of DevOps

- **Faster time to market** — release in hours or days instead of months.
- **Higher deployment frequency** — many small releases instead of one huge risky one.
- **Lower failure rate of new releases** — small changes are easier to test and easier to reverse.
- **Faster Mean Time To Recovery (MTTR)** — when something does break, fix or roll back quickly.
- **Better collaboration** — one team, one set of goals, shared tooling and shared metrics.

2.3 Benefits of DevOps

Benefit	What it means in practice
Speed	Automated pipelines remove the days lost to manual builds, manual testing and change-approval meetings.
Reliability	Every change passes the same automated tests, so quality does not depend on who was on duty.
Scalability	Infrastructure as Code lets you go from 3 servers to 300 by changing one number in a file.
Security	Security checks are built into the pipeline ("DevSecOps") instead of being a gate at the very end.
Cost saving	Fewer outages, less firefighting, less duplicated manual effort.
Improved culture	Shared responsibility and blameless post-mortems reduce friction and staff turnover.

2.4 The DevOps lifecycle

The lifecycle is drawn as an infinity loop (∞) on purpose: the work never stops and never restarts from zero. What you learn from monitoring in production feeds straight back into the next round of planning.

The DevOps Lifecycle — an infinite loop, not a straight line

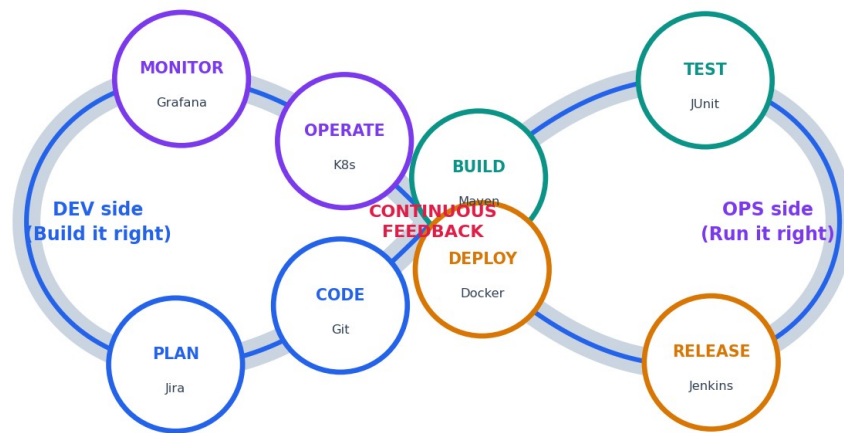


Figure 2.1 — The eight stages of the DevOps lifecycle and a typical tool for each

Stage	What happens	Typical tools
1. Plan	Requirements, user stories, sprint backlog, and the definition of done.	Jira, Azure Boards, Trello
2. Code	Developers write code and version it in a shared repository.	Git, GitHub, GitLab, VS Code
3. Build	Source is compiled and packaged into an artifact (.jar, .war, image).	Maven, Gradle, npm, Jenkins
4. Test	Automated unit, integration and security tests run against the build.	JUnit, Selenium, SonarQube
5. Release	A tested artifact is versioned and approved as ready to go out.	Jenkins, GitLab CI, Nexus
6. Deploy	The artifact is pushed to the target environment.	Docker, Ansible, Argo CD
7. Operate	The application runs; scaling, configuration and infrastructure are managed.	Kubernetes, Terraform, Ansible
8. Monitor	Metrics, logs and alerts are collected and fed back to Plan.	Prometheus, Grafana, ELK
MEMORY HOOK PC-BT-RD-OM → Plan, Code, Build, Test, Release, Deploy, Operate, Monitor. The first four are the Dev side of the loop; the last four are the Ops side.		

2.5 Roles and responsibilities of a DevOps engineer

A DevOps engineer is the person who builds and maintains the road that code travels on. Typical responsibilities:

- **Build and maintain CI/CD pipelines** — so every commit is automatically built, tested and deployed.
- **Automate infrastructure** — provision servers, networks and databases using code (Terraform, Ansible, CloudFormation).
- **Manage version control** — define the branching strategy, protect the main branch, review merges.
- **Containerise and orchestrate applications** — write Dockerfiles, manage Kubernetes manifests.

- **Set up monitoring, logging and alerting** — dashboards, alert rules, on-call runbooks.
- **Manage configuration and secrets** — keep environments consistent; never hard-code passwords.
- **Bake in security** — vulnerability scanning of images and dependencies inside the pipeline.
- **Incident response and post-mortems** — restore service fast, then find the systemic cause without blaming individuals.
- **Collaborate and document** — act as the bridge between developers, testers, security and operations.

2.6 The four DevOps principles

The four DevOps principles — remember them as CAMS

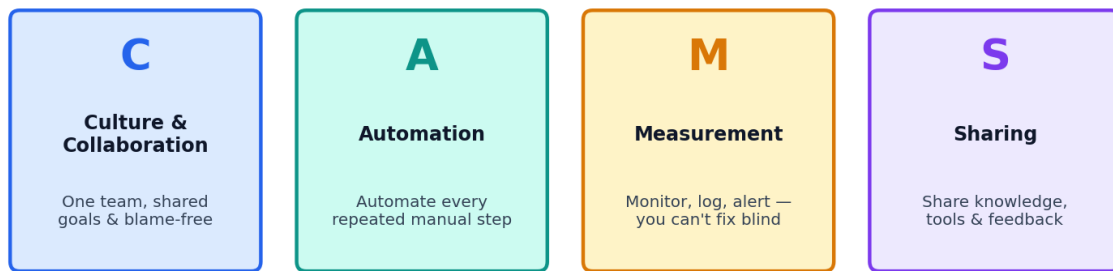


Figure 2.2 — CAMS: Culture, Automation, Measurement, Sharing

(a) Collaboration

Dev, Ops, QA and Security work as one team with shared goals, shared tools and shared metrics. Practically this means joint stand-ups, a shared chat channel, developers being on the on-call rota, and blameless post-mortems that ask "what in our system allowed this?" rather than "who did this?"

(b) Automation

The rule of thumb: if a human does it more than twice, automate it. Automate builds, tests, deployments, infrastructure provisioning, and even the alerting. Automation removes human error, makes the process repeatable, and frees engineers for work that actually needs thought.

(c) Continuous Improvement

Nothing is ever finished. Measure the pipeline itself — how long does a build take, how often do deployments fail, how long to recover — then attack the worst number. This is the Kaizen idea borrowed from lean manufacturing. The four industry-standard DORA metrics are: deployment frequency, lead time for changes, change failure rate, and mean time to restore.

(d) Monitoring (and Measurement)

You cannot improve what you cannot see. Collect metrics (numbers), logs (events) and traces (request journeys) from every layer, alert on the things that hurt users, and feed what you learn back into planning.

2.7 Agile vs DevOps

A very common exam question. The key line: Agile ends where DevOps begins — Agile optimises how software is developed, DevOps optimises how it is developed, delivered and operated.

Point of comparison	Agile	DevOps
Main focus	Development process — building the right thing, iteratively.	Whole delivery chain — build, release and run it reliably.

Point of comparison	Agile	DevOps
People involved	Developers, product owner, testers.	Developers, operations, QA, security — everyone.
Goal	Adapt quickly to changing requirements.	Deliver continuously and keep the system healthy in production.
Ends at	Working software at the end of a sprint.	Software running successfully in front of real users.
Feedback source	Customer / product owner review.	Customers plus production monitoring and metrics.
Key practices	Sprints, stand-ups, backlog, retrospectives.	CI/CD, IaC, containerisation, monitoring, automation.
Release size	Iteration / sprint-sized.	Very small and very frequent — often many per day.
Relationship	A software development methodology.	A culture and practice that extends Agile to operations.
ONE-LINE ANSWER Agile makes development fast. DevOps makes delivery and operations fast too. They are partners, not rivals — most DevOps teams are also Agile teams.		

2.8 Infrastructure as Code (IaC)

Definition: Infrastructure as Code is the practice of defining and managing infrastructure — servers, networks, load balancers, databases — in machine-readable configuration files, which are stored in version control and applied automatically, instead of being set up by hand.

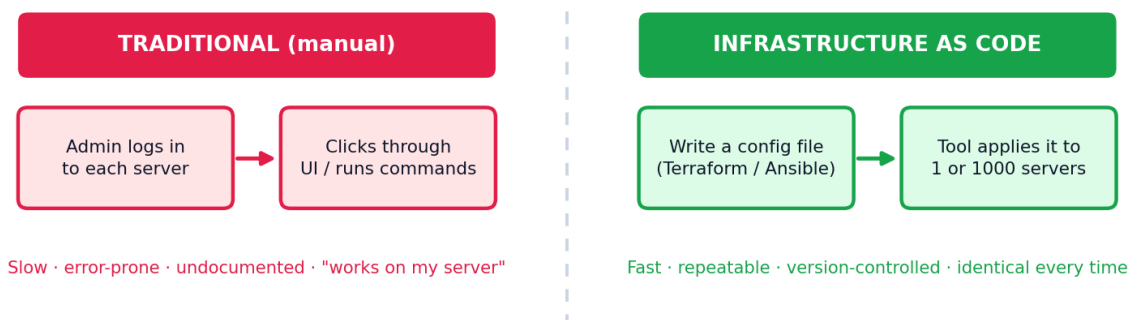


Figure 2.3 — Manual infrastructure vs Infrastructure as Code

Why it matters

- **Repeatable** — the same file produces the same environment every single time, so dev, test and production actually match.
- **Versioned** — infrastructure lives in Git, so you get history, code review and the ability to roll back.
- **Fast and scalable** — changing `count = 3` to `count = 30` builds 30 servers.
- **Self-documenting** — the file is the documentation, and it can never drift out of date.
- **Kills configuration drift** — no more "server 4 is slightly different and nobody knows why".

Two styles of IaC

Style	You write...	Example tools
Declarative (functional)	The desired end state. The tool figures out the steps.	Terraform, CloudFormation, Kubernetes YAML
Imperative (procedural)	The exact sequence of commands to reach that state.	Shell scripts, older Chef recipes

Related idea: **provisioning** tools (Terraform) create the infrastructure; **configuration management** tools (Ansible, Chef, Puppet) install and configure software on it.

Example — Terraform (declarative)

```
# main.tf - create two identical web servers on AWS
resource "aws_instance" "web" {
  count          = 2
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server-${count.index}"
  }
}

# Apply it:
terraform init    # download the AWS provider
terraform plan    # show exactly what will change - no surprises
terraform apply   # actually build it
terraform destroy # tear it all down again
```

Example — Ansible (configuration management)

```
# install-nginx.yml
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Make sure nginx is running and starts on boot
      service:
        name: nginx
        state: started
        enabled: yes

# Run it against every host in the 'webservers' group:
ansible-playbook -i inventory install-nginx.yml
```

IDEMPOTENCY — REMEMBER THIS WORD

An IaC script is idempotent: running it once or running it fifty times gives the same result. Ansible checks "is nginx already installed?" before acting. This is why IaC is safe to re-run, and it is a favourite one-mark

question.

Unit 3 — Version Control Systems

3.1 What a Version Control System is

Definition: A Version Control System (VCS) is a tool that records every change made to a set of files over time, so you can see who changed what and when, go back to any earlier version, and let many people work on the same project without overwriting each other.

Without a VCS you end up with folders called `project_final`, `project_final_v2`, `project_final_REALLY_final` — and no idea which one is live. A VCS replaces all of that with one repository and a clean history.

What a VCS gives you

- **History** — every version is kept; you can diff any two points in time.
- **Traceability** — every change has an author, a timestamp and a message explaining why.
- **Collaboration** — many developers work in parallel; the VCS merges their work.
- **Branching** — experiment safely without touching the working version.
- **Rollback** — a bad change can be reverted in seconds.
- **Backup** — in a distributed VCS, every clone is a full backup of the project.

3.2 Types of Version Control Systems

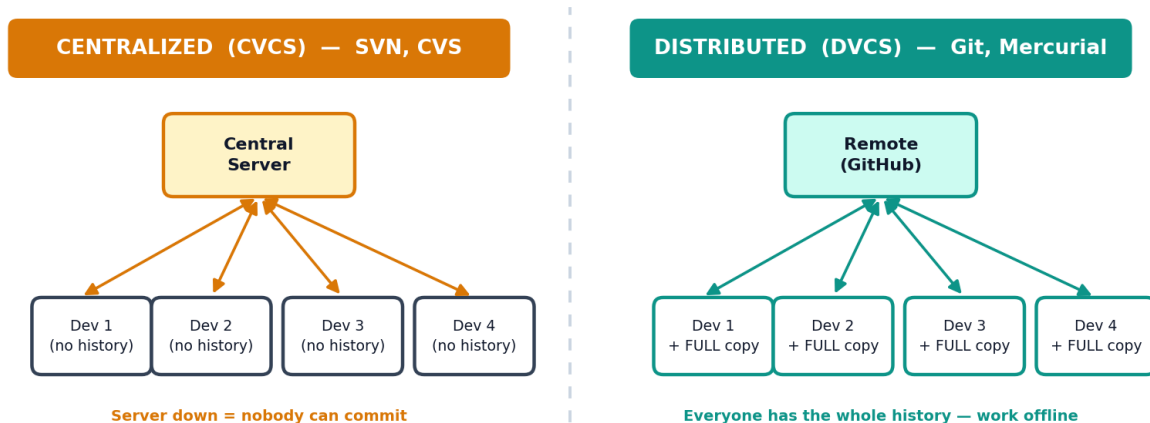


Figure 3.1 — Centralized vs Distributed version control

Type	How it works	Pros	Cons	Examples
Local VCS	History kept in a database on your own machine only.	Simple.	No collaboration; one disk failure loses everything.	RCS, SCCS
Centralized (CVCS)	One central server holds the full history; developers check out a working copy.	Easy to administer; clear single source of truth.	Server is a single point of failure; no offline commits; slow operations.	SVN, CVS, Perforce
Distributed (DVCS)	Every developer clones the entire repository including full history.	Works offline; fast; every clone is a backup; cheap branching.	Bigger initial clone; slightly steeper learning curve.	Git, Mercurial, Bazaar

3.3 Git fundamentals

Git is a free, open-source distributed version control system created by Linus Torvalds in 2005 to manage the Linux kernel. It is now the default choice almost everywhere.

Key characteristics

- **Distributed** — your clone contains the complete history, so almost every operation is local and instant.
- **Snapshots, not diffs** — Git stores a picture of all files at each commit, not a list of line changes. This makes branching and switching very fast.
- **Integrity** — every object is identified by a SHA-1 hash, so silent corruption is impossible to miss.
- **Cheap branching** — a branch is just a 40-character pointer to a commit, so creating one costs nothing.

Git vs GitHub — do not mix these up

Git	GitHub
The tool. Software installed on your computer.	A website. A hosting service for Git repositories.
Created by Linus Torvalds, 2005.	Launched 2008, now owned by Microsoft.
Works completely offline.	Needs the internet.
Does version control.	Adds pull requests, issues, code review, CI (Actions), access control.

Alternatives to GitHub with the same role: **GitLab**, **Bitbucket**, **Azure Repos**.

3.4 Git repository concepts — the four areas

Almost every Git command makes sense once you know these four places. A change moves left to right as it becomes more permanent.

The four areas of Git — every command just moves changes between them

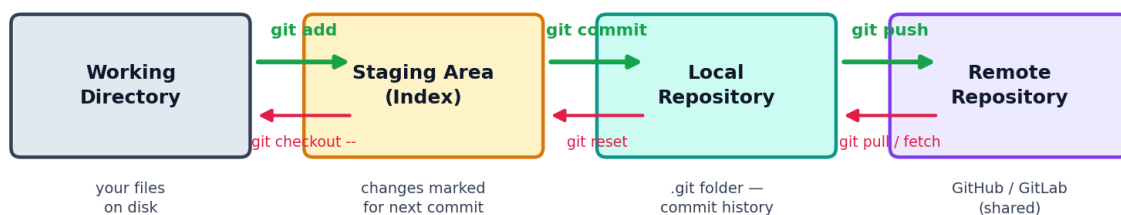


Figure 3.2 — Working Directory → Staging Area → Local Repository → Remote Repository

Area	What lives there	How to get there
Working Directory	The actual files you edit. Changes here are untracked or modified.	Just edit a file
Staging Area (Index)	A rehearsal space. Changes you have chosen to include in the next commit.	git add
Local Repository	The .git folder — the permanent, committed history on your machine.	git commit

Area	What lives there	How to get there
Remote Repository	The shared copy on GitHub/GitLab that the team pulls from and pushes to.	git push
WHY A STAGING AREA? It lets you split messy work into clean commits. If you fixed a bug and also changed the README, you can stage and commit them separately, so the history stays readable and either can be reverted alone.		

Other terms you must know

- **Repository (repo)** — the project folder plus its full history (the hidden `.git` directory).
- **Commit** — one saved snapshot, with a unique hash, an author, a timestamp and a message.
- **HEAD** — a pointer to the commit you are currently on (normally the tip of the current branch).
- **Branch** — a movable pointer to a commit; an independent line of development.
- **origin** — the default nickname Git gives to the remote you cloned from.
- **.gitignore** — a file listing paths Git should never track, e.g. `node_modules/`, `*.log`, `.env`.

3.5 Essential Git commands

Setup (do this once per machine)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list          # check what is configured
```

The commands in the syllabus

Command	What it does	Example
git init	Creates a brand-new empty repository in the current folder (makes <code>.git</code>).	git init
git clone	Downloads an existing remote repository, with its full history, to your machine.	git clone https://github.com/user/app.git
git add	Moves changes from the working directory into the staging area.	git add index.html / git add .
git commit	Permanently records the staged changes into the local repository.	git commit -m "Add login validation"
git push	Uploads your local commits to the remote repository.	git push origin main
git pull	Downloads changes from the remote and merges them into your branch. (= fetch + merge)	git pull origin main
git branch	Lists, creates or deletes branches.	git branch feature-login
git merge	Joins another branch's history into your current branch.	git merge feature-login

Very useful supporting commands

```
git status          # what is modified / staged? Use this constantly
git log --oneline --graph  # compact visual history
```

```

git diff                # unstaged changes
git diff --staged        # staged changes
git checkout -b new-feature # create a branch AND switch to it
git switch main          # newer, clearer way to change branch
git remote -v            # show the remote URLs
git fetch origin          # download remote changes WITHOUT merging
git restore file.txt      # throw away local changes to a file
git reset --soft HEAD~1  # undo last commit, keep the changes staged
git revert <hash>         # safely undo a commit by making a new one
git stash                # park uncommitted work temporarily
git stash pop             # bring it back

```

A complete worked example

Starting a project, making the first commit, and connecting it to GitHub:

```

mkdir student-portal && cd student-portal
git init                                # -> Initialized empty Git repository
echo "# Student Portal" > README.md
git status                              # README.md shown as untracked (red)
git add README.md                       # now staged (green)
git commit -m "Initial commit: add README"

git remote add origin https://github.com/you/student-portal.git
git branch -M main
git push -u origin main                 # -u remembers the link for next time

# Later, day-to-day rhythm:
git pull origin main                   # get the team's latest work FIRST
# ...edit files...
git add .
git commit -m "Add attendance module"
git push origin main

```

GOOD COMMIT MESSAGES

Use the imperative mood and say WHY, not what: "Fix crash when student ID is empty" beats "changes" or "update file". Your future self reading git log will thank you.

3.6 Branching and merging

A **branch** is an independent line of development. You create one so that unfinished or risky work never destabilises the working version. The default branch is normally called **main** (older repositories call it **master**).

Branching & merging — the heart of team workflow

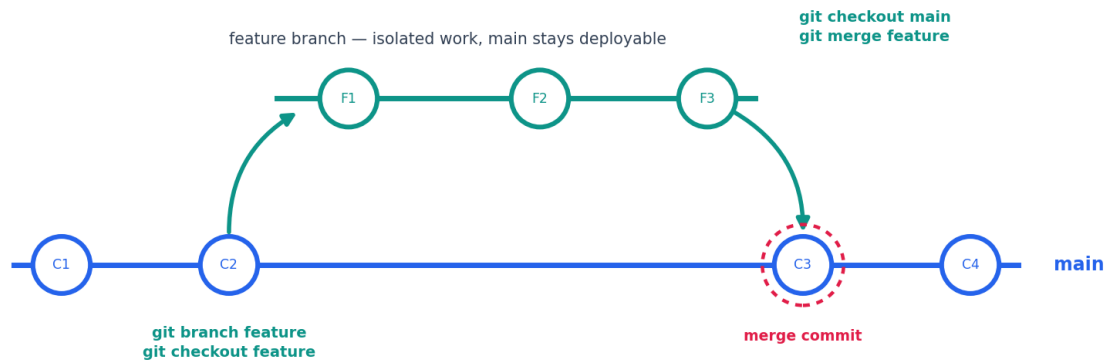


Figure 3.3 — Creating a feature branch and merging it back into main

The workflow

```
git checkout main
git pull origin main           # start from the latest code
git checkout -b feature-payment # 1. branch off

# ...work, commit, work, commit on the branch...
git add . && git commit -m "Add payment gateway"
git push origin feature-payment # share it / open a Pull Request

git checkout main              # 2. go back to main
git merge feature-payment      # 3. merge the work in
git push origin main
git branch -d feature-payment  # 4. delete the finished branch
```

Two kinds of merge

Merge type	When it happens	Result
Fast-forward	main has not moved since you branched off.	Git simply slides the main pointer forward. No merge commit is created.
Three-way merge	Both main and your branch gained new commits.	Git combines both histories and creates a new merge commit with two parents.

Merge conflicts

A conflict happens when two branches change the same lines of the same file. Git cannot guess which version is right, so it stops and asks you. The file will contain markers:

```
<<<<<<< HEAD
  price = 100;           // the version on your current branch
=====
  price = 150;           // the version coming from the other branch
>>>>>>> feature-payment
```

To resolve it:

4. Open the file and edit it so only the correct final code remains.
5. Delete all three marker lines (<<<<<<<, =====, >>>>>>>).

6. `git add <file>` — this tells Git the conflict is resolved.

7. `git commit` — completes the merge.

AVOIDING CONFLICTS

Pull often, keep branches short-lived, and keep each branch focused on one feature. A branch that lives for three weeks will conflict; one that lives for a day usually will not.

Common branching strategies

Strategy	Idea	Suits
Git Flow	Long-lived main + develop branches, plus feature, release and hotfix branches.	Scheduled releases, versioned products
GitHub Flow	One main branch; every change is a short feature branch merged via Pull Request.	Web apps deploying continuously
Trunk-based	Everyone commits to main many times a day behind feature flags.	Mature CI/CD teams

3.7 Repository management

- **Protect the main branch** — block direct pushes; require a Pull Request and at least one approving review.
- **Use Pull / Merge Requests** — they are where code review, automated CI checks and discussion happen before code lands.
- **Tag releases** — mark a commit as v1.0, v1.1 and so on, so you can always find exactly what shipped.
- **Keep a good .gitignore** — never commit build output, dependencies, or secrets.
- **Never commit credentials** — once a password is in Git history it is effectively public. Use environment variables or a secret manager.
- **Write a README** — what the project is, how to install it, how to run it, how to contribute.
- **Choose public vs private deliberately** and use teams/roles to control who can write.

```
git tag -a v1.0 -m "First production release"
git push origin v1.0
```

```
# .gitignore example
node_modules/
*.log
.env
target/
__pycache__/
```

Unit 4 — Continuous Integration & Jenkins

4.1 What Continuous Integration is

Definition: Continuous Integration is the practice where developers merge their code into a shared repository frequently — ideally several times a day — and every merge automatically triggers a build and an automated test run, so that integration problems are found within minutes instead of months.

The problem CI solves

Without CI, five developers work in isolation for a month and then merge on the same afternoon. Nothing compiles, nobody knows whose change broke it, and the team loses a week. This is called integration hell. CI removes it by making integration a tiny, constant, boring event rather than a rare, huge, terrifying one.

Benefits of CI

- **Bugs found early** — and a bug found 10 minutes after you wrote it costs a fraction of one found in production.
- **No integration hell** — small merges rarely conflict badly.
- **Always a working build** — the repository is never broken for long.
- **Fast feedback** — the developer who broke it is told immediately, while the change is still fresh.
- **Confidence to refactor** — the test suite is a safety net.
- **Consistent builds** — built by a machine in a clean environment, so "it works on my laptop" stops being an argument.

4.2 CI workflow

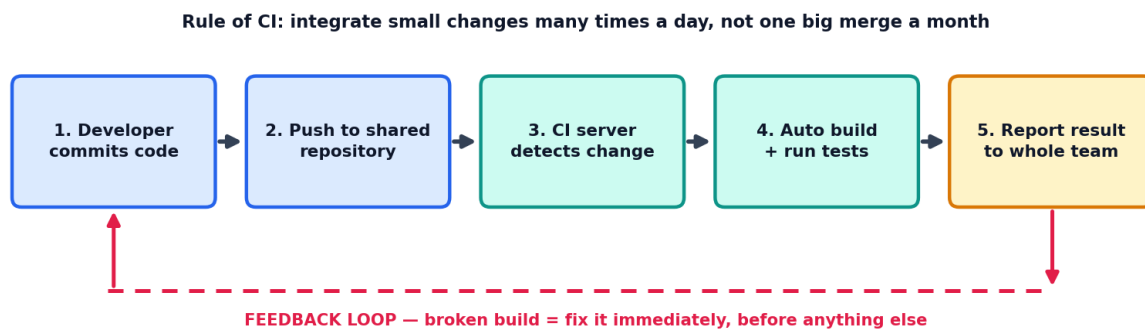


Figure 4.1 — The Continuous Integration cycle

- 8. Commit** — a developer commits a small, self-contained change locally.
- 9. Push** — the change is pushed to the shared repository (or opened as a Pull Request).
- 10. Trigger** — the CI server notices, either through a webhook (instant, preferred) or by polling the repo on a schedule.
- 11. Checkout & Build** — the CI server pulls the latest code onto a clean agent and compiles/packages it.
- 12. Test** — unit tests, then integration tests, then static code analysis and security scans.
- 13. Report** — results go to the team: dashboard, email, Slack, or a status check on the Pull Request.
- 14. Fix immediately** — if the build is red, fixing it becomes the team's top priority. Nobody builds on a broken foundation.

THE GOLDEN RULES OF CI

(1) Commit to the shared branch at least daily. (2) Every commit triggers a build. (3) Keep the build fast — under 10 minutes. (4) Never go home on a red build. (5) Fix a broken build before writing any new code.

4.3 CI tools overview

Tool	Type	Notes
Jenkins	Self-hosted, open source	The most widely taught. Huge plugin ecosystem (1800+), highly flexible, but you maintain the server yourself.
GitLab CI/CD	Built into GitLab	Configured with a .gitlab-ci.yml file in the repo. No separate server to install.
GitHub Actions	Built into GitHub	YAML workflows in .github/workflows/. Enormous marketplace of ready-made actions.
CircleCI / Travis CI	Cloud SaaS	Quick to start, minimal maintenance, usage-based pricing.
Azure DevOps Pipelines	Cloud / self-hosted	Strong Microsoft and enterprise integration.
TeamCity / Bamboo	Commercial	JetBrains and Atlassian offerings, integrate well with their other tools.

4.4 Introduction to Jenkins

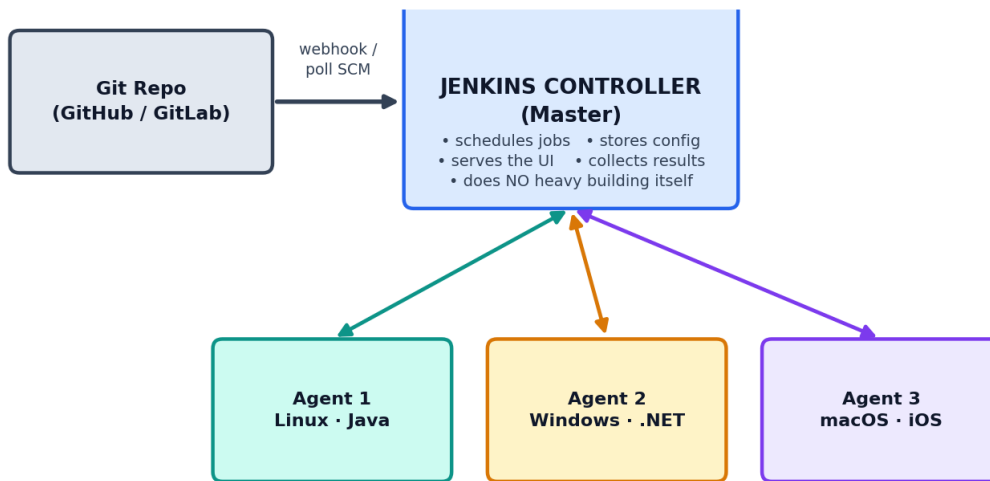
Jenkins is a free, open-source automation server written in Java. It is the engine that watches your repository and runs your build, test and deployment steps. It began life as Hudson in 2004 and was renamed Jenkins in 2011.

Why Jenkins is so widely used

- **Open source and free.**
- **Plugins for everything** — Git, Docker, Kubernetes, Maven, Slack, AWS: over 1,800 of them.
- **Platform independent** — runs anywhere Java runs; agents can be Linux, Windows or macOS.
- **Distributed builds** — scale out by adding agents.
- **Pipeline as code** — the whole pipeline lives in a Jenkinsfile in your repository.
- **Large community** — almost every problem you hit has already been answered.

4.5 Jenkins architecture (Controller-Agent)

Jenkins Controller-Agent architecture



Agents (Nodes) do the actual build work — add more to scale out

Figure 4.2 — Jenkins Controller (Master) with multiple Agents (Nodes)

Component	Role
Controller (Master)	The brain. Serves the web UI, stores all job configuration, schedules builds, decides which agent runs which job, and collects the results. It should not run heavy builds itself.
Agent (Node / Slave)	The muscle. A separate machine (or container) that actually checks out code and executes the build steps. Agents can have different OSs and toolchains.
Executor	One build slot on an agent. An agent with 4 executors can run 4 builds at the same time.
Job / Project	One configured unit of work — e.g. "build the payments service".
Build	One single execution of a job, numbered #1, #2, #3...
Workspace	The directory on the agent where the code is checked out and the build happens.

WHY SEPARATE CONTROLLER AND AGENTS?

Scalability (add agents to build more in parallel), isolation (a build crash does not take down Jenkins), and platform coverage (build a Windows app on a Windows agent and an iOS app on a macOS agent from the same controller).

4.6 Jenkins dashboard overview

When you log in to Jenkins (default <http://localhost:8080>) you land on the dashboard. The parts you need to know:

Element	What it shows / does
Job list	Every job, with its last success, last failure and last build duration.
Weather icon	A sun-to-storm icon summarising the recent build health of that job.
Build status ball	Blue/green = success, red = failed, yellow = unstable (built but tests failed), grey = not built.
New Item	Create a job — Freestyle project, Pipeline, Multibranch Pipeline, Folder.

Element	What it shows / does
Build History / Build Queue	Recent builds, and what is waiting for a free executor.
Manage Jenkins	System configuration, Global Tool Configuration, Plugin Manager, Credentials, Nodes, Security.
Console Output	The full log of a single build — the first place you look when something fails.

4.7 Jenkins jobs — Freestyle vs Pipeline

	Freestyle project	Pipeline
How you configure it	Click through web forms in the UI.	Write code in a Jenkinsfile.
Stored where	Inside Jenkins' own config (XML on the controller).	In your Git repository, next to the code.
Version controlled?	No — changes are not tracked.	Yes — reviewed and versioned like any code.
Complex logic	Very limited; loops and conditions are painful.	Full Groovy: loops, conditions, parallel stages, try/catch.
Survives a Jenkins rebuild?	Only if you backed up Jenkins.	Yes — it is in the repo.
Visual stage view	No.	Yes — each stage shown with its own timing.
Best for	Small, simple, one-off tasks; learning.	Real CI/CD. This is the modern standard.

Two flavours of Pipeline syntax

- **Declarative** — newer, structured, starts with `pipeline { }`. Easier to read, has built-in error handling. Use this by default.
- **Scripted** — older, starts with `node { }`, full Groovy freedom. Use only when you need something declarative cannot express.

4.8 Jenkinsfile basics

A **Jenkinsfile** is a text file in the root of your repository that defines the whole pipeline as code. Because it sits in Git, the pipeline is reviewed, versioned and rolled back exactly like application code.

Skeleton of a Declarative Pipeline

```

pipeline {
    agent any                // run on any available agent

    tools {
        maven 'Maven-3.9'    // named in Global Tool Configuration
    }

    environment {
        APP_NAME = 'student-portal'
    }
}
```

```

stages {
    stage('Checkout') {
        steps {
            git branch: 'main', url: 'https://github.com/you/app.git'
        }
    }

    stage('Build') {
        steps {
            sh 'mvn clean package -DskipTests'
        }
    }

    stage('Test') {
        steps {
            sh 'mvn test'
        }
        post {
            always {
                junit 'target/surefire-reports/*.xml' // publish results
            }
        }
    }

    stage('Deploy') {
        when { branch 'main' } // only deploy from main
        steps {
            sh './deploy.sh'
        }
    }
}

post {
    success { echo "${APP_NAME} build ${env.BUILD_NUMBER} passed" }
    failure { mail to: 'team@example.com', subject: 'Build FAILED', body: 'Check
Jenkins.' }
    always { cleanWs() } // clean the workspace
}
}

```

The building blocks

Block	Purpose
pipeline { }	Wraps the whole Declarative Pipeline. Mandatory outermost block.
agent	Where to run: any, none, label 'linux', or docker { image 'maven:3.9' }.
stages { }	Container for all the stages.
stage('Name') { }	One logical phase — Build, Test, Deploy. Each gets its own box in the visual view.
steps { }	The actual commands inside a stage (sh, bat, echo, git...).

Block	Purpose
environment { }	Environment variables available to every stage.
tools { }	Auto-installs and adds tools (Maven, JDK, NodeJS) to the PATH.
when { }	A condition — run this stage only if it is true.
post { }	Actions after a stage or the whole pipeline: always, success, failure, unstable, changed.
parallel { }	Run several stages at the same time to save build time.

4.9 Build automation concepts

Build automation is scripting the whole process of turning source code into a runnable artifact: fetching dependencies, compiling, running static checks, packaging, versioning and publishing.

Language / stack	Build tool	Command that packages it
Java	Maven, Gradle, Ant	mvn clean package → target/app.jar
JavaScript / Node	npm, yarn, Vite, webpack	npm run build → dist/
Python	pip, Poetry, setuptools	python -m build → dist/*.whl
.NET	MSBuild, dotnet CLI	dotnet publish
C / C++	Make, CMake	make

Two related terms: an **artifact** is the packaged output of a build (a .jar, a .zip, a Docker image); an **artifact repository** (Nexus, JFrog Artifactory, Docker Hub) is where those artifacts are stored and versioned so a deployment can fetch the exact build that was tested.

BUILD ONCE, DEPLOY MANY

A cornerstone rule. Build the artifact exactly once, then promote that identical artifact through test → staging → production. If you rebuild for each environment you can no longer prove that what you tested is what you shipped.

4.10 Automated testing in the CI pipeline

Tests are what make CI trustworthy. Arrange them as a pyramid: many fast cheap tests at the bottom, few slow expensive ones at the top.

Level	What it checks	Speed	Tools
Unit tests	One function or class in isolation, with dependencies mocked.	Milliseconds — thousands run in seconds	JUnit, pytest, Jest
Integration tests	Several components together — service + real database, or two APIs.	Seconds to minutes	TestNG, Postman/Newman, Testcontainers
Static analysis / security	Code smells, coverage thresholds, known-vulnerable dependencies.	Seconds	SonarQube, Trivy, OWASP Dep-Check
UI / End-to-end tests	A real user journey through the running application.	Minutes — run fewer of these	Selenium, Cypress, Playwright

Level	What it checks	Speed	Tools
Performance tests	Behaviour under load.	Minutes to hours — often nightly	JMeter, k6, Gatling

In Jenkins, plug results back into the UI with **junit** for test reports, **publishHTML** for coverage, and quality gates that fail the build if coverage drops below a threshold.

4.11 Continuous feedback in the CI process

A pipeline that runs but tells nobody is useless. Continuous feedback means every relevant person learns the result of every change, fast, through the channel they already use.

- **Build status in the tool** — red/green balls, the stage view showing exactly which stage failed.
- **Notifications** — Slack, Microsoft Teams or email on failure (and on recovery).
- **Pull Request checks** — the PR shows a green tick or a red cross so bad code cannot be merged.
- **Test and coverage reports** — published as part of the build, trended over time.
- **Production monitoring** — the final feedback loop: metrics and alerts tell you how the change actually behaves for real users.

FEEDBACK MUST BE FAST, RELEVANT AND ACTIONABLE

Fast: within minutes. Relevant: tell the person who made the change, not everyone. Actionable: say which test failed and link to the log — "build failed" alone helps nobody. Beware alert fatigue: if people start ignoring notifications, the feedback loop is already broken.

Unit 5 — Continuous Deployment & Containerization

5.1 Continuous Delivery vs Continuous Deployment

Both are abbreviated CD, both come after CI, and the difference is exactly one thing: whether a human presses a button.

The CI/CD pipeline — and where Delivery ends and Deployment begins

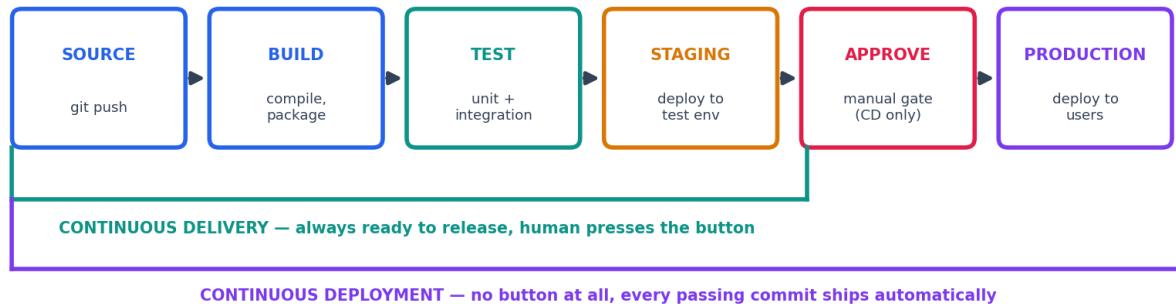


Figure 5.1 — Where Continuous Delivery ends and Continuous Deployment begins

	Continuous Delivery	Continuous Deployment
Definition	Every change that passes the pipeline is automatically made ready to release.	Every change that passes the pipeline is automatically released to production.
Final step to production	Manual — someone approves and clicks deploy.	Automatic — no human involved.
Release decision	A business decision.	Made by the pipeline; the tests are the gate.
Risk level	Lower — there is a human checkpoint.	Higher — demands excellent tests, monitoring and instant rollback.
Suits	Regulated industries, mobile apps, large enterprises.	Mature web-scale teams (Netflix, Amazon, Etsy).
ONE-LINE DISTINCTION Continuous Delivery = always READY to deploy. Continuous Deployment = always DOES deploy. Every Continuous Deployment setup is also doing Continuous Delivery; the reverse is not true.		

5.2 The CI/CD pipeline as a whole

Stage	What happens	Fails if...
Source	Commit/PR triggers the pipeline via webhook.	—
Build	Compile and package into a versioned artifact / image.	Compilation error, missing dependency
Test	Unit, integration, static analysis, security scan.	A test fails or coverage drops
Release	Artifact is tagged and pushed to a registry / repository.	Registry unreachable, tag conflict
Deploy to staging	Deployed to a production-like environment; smoke tests run.	Smoke tests fail

Stage	What happens	Fails if...
Approval gate	Human approval (Continuous Delivery only).	Not approved
Deploy to production	Released to users using a safe strategy.	Health checks fail → automatic rollback
Monitor	Metrics, logs and alerts confirm the release is healthy.	Error rate spikes → roll back

5.3 Deployment strategies

How you replace the old version with the new one determines how much your users notice, and how quickly you can undo a mistake.

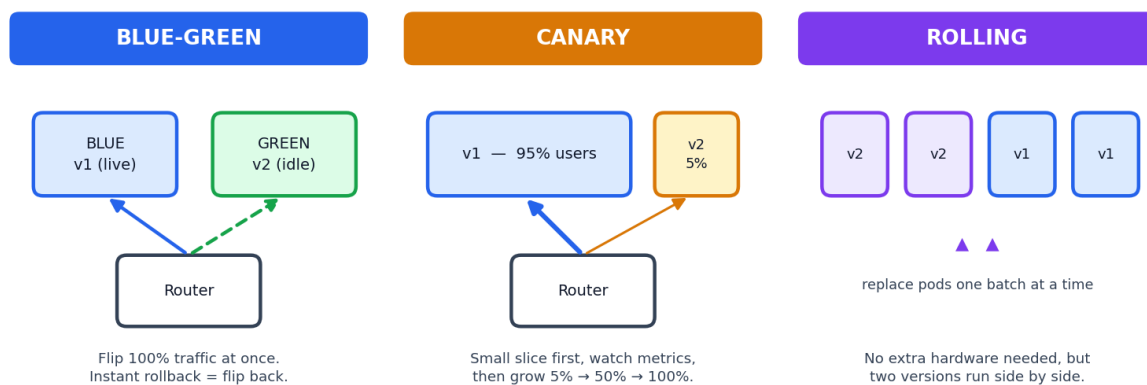


Figure 5.2 — Blue-Green, Canary and Rolling deployments

Strategy	How it works	Advantages	Disadvantages
Recreate (Big Bang)	Stop the old version completely, then start the new one.	Simplest; no version mixing.	Downtime for every release.
Rolling update	Replace instances a few at a time until all are new.	No downtime; no extra hardware. Kubernetes default.	Old and new run together for a while; slower rollback.
Blue-Green	Two identical environments. Green gets v2, then all traffic switches at once.	Zero downtime; rollback is instantly flipping back to Blue.	Needs double the infrastructure; database changes are tricky.
Canary	Send a small % of users to v2, watch metrics, then increase gradually.	Real-user testing with tiny blast radius; catches issues early.	More complex routing and monitoring needed; slower rollout.
A/B testing	Route specific user segments to different versions.	Measures business impact, not just technical health.	Not a safety strategy — it is a product experiment.
Shadow / Mirroring	Copy real traffic to v2 without returning its responses to users.	Zero user risk; tests with production-like load.	Expensive; side effects must be carefully suppressed.

5.4 Configuration management

Definition: Configuration management is the practice of establishing and maintaining consistent settings, software and state across all your servers and environments — automatically, and from a definition kept in version control.

The problem it solves — configuration drift

Servers that started identical slowly diverge as people apply manual fixes. Eventually server 7 behaves differently from servers 1–6 and nobody knows why. Configuration management makes the definition file the single source of truth and re-applies it, so drift is corrected automatically.

Tool	Language	Model	Agent needed?
Ansible	YAML playbooks	Push — control machine connects out over SSH	No (agentless)
Puppet	Puppet DSL	Pull — agent asks the master for its config	Yes
Chef	Ruby recipes	Pull	Yes
SaltStack	YAML	Push or pull	Optional

Also key: keep configuration **out of the code**. The same artifact should run in dev, staging and production, with only environment variables or config files differing. Secrets belong in a secret manager (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets) — never in Git.

5.5 Virtualization vs Containerization

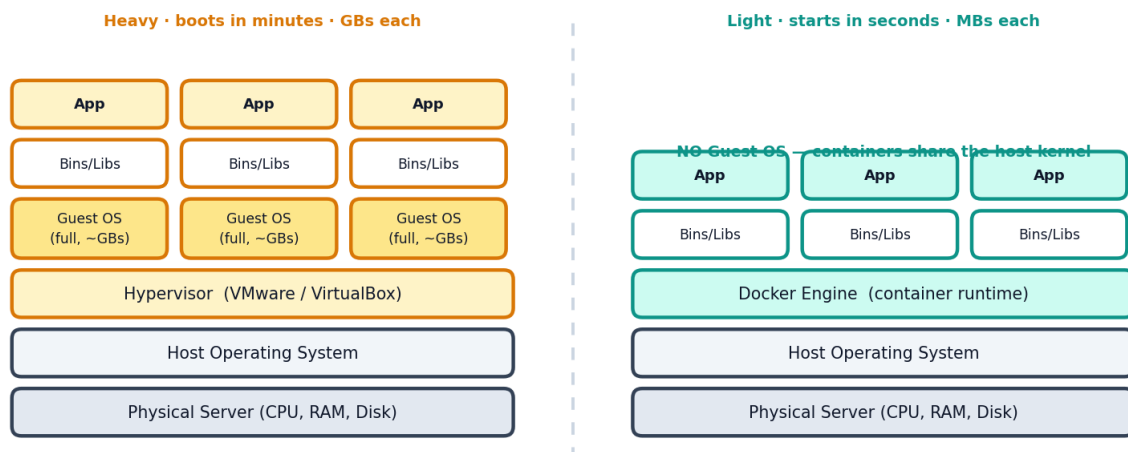


Figure 5.3 — Virtual machines each carry a full guest OS; containers share the host kernel

Aspect	Virtual Machines	Containers
What is virtualised	The hardware.	The operating system.
Guest OS	Each VM has its own full OS.	None — all containers share the host kernel.
Size	Gigabytes.	Megabytes.
Start-up time	Minutes.	Seconds or less.
Isolation	Very strong (hardware level).	Good (process level), but weaker than a VM.
Density per host	Tens.	Hundreds or thousands.
Managed by	Hypervisor — VMware ESXi, Hyper-V, KVM, VirtualBox.	Container engine — Docker, containerd, Podman.
Best for	Running different OSs; strict isolation; legacy	Microservices, CI/CD, fast scaling, portable

Aspect	Virtual Machines	Containers
	monoliths.	apps.
THEY ARE NOT ENEMIES In real cloud deployments you almost always run containers inside virtual machines: the cloud provider gives you VMs, and Kubernetes packs containers onto them. You get the VM's strong isolation and the container's speed and density.		

5.6 Docker basics

Docker is the platform that made containers mainstream. It packages an application together with everything it needs to run — code, runtime, libraries, environment variables and configuration — into a single portable unit called an image.

The vocabulary

Term	Meaning
Dockerfile	A text recipe describing how to build an image, one instruction per line.
Image	A read-only, layered template built from a Dockerfile. Think 'class'.
Container	A running instance of an image, with a thin writable layer on top. Think 'object'.
Registry	Storage for images — Docker Hub, AWS ECR, GitLab Registry, Harbor.
Volume	Storage that lives outside the container so data survives when the container is deleted.
Docker Compose	A tool to define and run a multi-container application from one docker-compose.yml file.
Layer	Each Dockerfile instruction adds a cached layer, which is why rebuilds are fast.
IMAGE : CONTAINER :: CLASS : OBJECT One image can produce hundreds of running containers. The image never changes; each container gets its own writable layer.	

5.7 Docker architecture

Docker architecture — Client, Host (Daemon), Registry

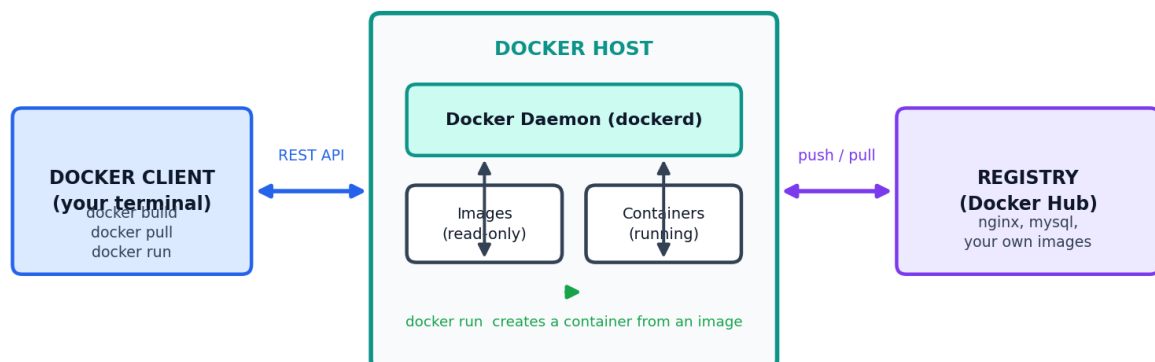


Figure 5.4 — Docker Client, Docker Host (daemon) and Registry

- **Docker Client** — the `docker` command you type. It sends your instructions to the daemon over a REST API.

- **Docker Daemon (dockerd)** — the background service on the Docker Host. It builds images, and creates, runs and destroys containers.
- **Docker Host** — the machine where the daemon, the images and the containers live.
- **Registry** — the remote store of images. **docker pull** downloads from it, **docker push** uploads to it. Docker Hub is the public default.
- **Docker Objects** — images, containers, networks and volumes.

5.8 Docker commands

Images

```
docker pull nginx:1.25      # download an image from a registry
docker images               # list local images
docker build -t myapp:1.0 .  # build an image from the Dockerfile in .
docker rmi myapp:1.0        # remove an image
docker tag myapp:1.0 user/myapp:1.0
docker push user/myapp:1.0   # upload to a registry
docker history myapp:1.0     # see the layers
```

Containers

```
docker run -d -p 8080:80 --name web nginx
# -d      run detached (in the background)
# -p 8080:80 map host port 8080 -> container port 80
# --name   give the container a readable name

docker ps                  # running containers
docker ps -a               # ALL containers, including stopped
docker stop web            # graceful stop
docker start web           # start it again
docker restart web
docker rm web              # delete a stopped container
docker logs -f web         # follow the container's output
docker exec -it web bash   # open a shell INSIDE the running container
docker inspect web         # full JSON details
docker stats               # live CPU / memory usage
docker cp web:/etc/nginx/nginx.conf . # copy a file out
```

Volumes, networks and cleanup

```
docker volume create mydata
docker run -d -v mydata:/var/lib/mysql mysql:8
docker network create mynet
docker run -d --network mynet --name db mysql:8

docker system prune -a      # delete unused images/containers/networks
docker-compose up -d        # start the whole multi-container app
docker-compose down         # stop and remove it
```

Worked example — Dockerfile for a Node.js app

```
# 1. Start from a small official base image
FROM node:20-alpine

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy dependency files FIRST so this layer is cached
COPY package*.json ./
RUN npm ci --omit=dev

# 4. Now copy the rest of the source
COPY . .

# 5. Document the port the app listens on
EXPOSE 3000

# 6. The command that runs when the container starts
CMD ["node", "server.js"]

docker build -t student-portal:1.0 .
docker run -d -p 3000:3000 --name portal student-portal:1.0
curl http://localhost:3000      # -> the app responds
```

WHY COPY PACKAGE.JSON BEFORE THE SOURCE?

Docker caches every layer. Dependencies change rarely but source changes constantly. Copying dependencies first means npm install is re-run only when package.json actually changes — turning a 90-second rebuild into a 3-second one.

Key Dockerfile instructions

Instruction	Purpose
FROM	The base image to build on. Always the first instruction.
WORKDIR	Sets the working directory for the instructions that follow.
COPY / ADD	Copies files from your machine into the image. Prefer COPY.
RUN	Executes a command at BUILD time (e.g. install packages) and creates a layer.
CMD	The default command at RUN time. Can be overridden on the command line.
ENTRYPOINT	The fixed executable at RUN time; CMD then supplies its default arguments.
ENV	Sets an environment variable inside the image.
EXPOSE	Documents which port the app listens on (does not publish it — -p does that).
VOLUME	Declares a mount point for persistent data.

Unit 6 — Kubernetes Fundamentals & Monitoring

6.1 Introduction to Kubernetes

Definition: Kubernetes (often written K8s — K, eight letters, s) is an open-source container orchestration platform that automates the deployment, scaling, networking, and management of containerised applications across a cluster of machines.

It was built by Google, based on their internal system Borg, and released as open source in 2014. It is now maintained by the Cloud Native Computing Foundation (CNCF).

Why Docker alone is not enough

Docker runs a container beautifully — on one machine. Now imagine 200 containers across 20 servers. Who decides which server each container runs on? Who restarts one that crashes at 3 a.m.? Who adds five more when traffic spikes, and removes them when it drops? Who routes traffic to them as their IP addresses keep changing? Who rolls out a new version without downtime? That is orchestration, and that is Kubernetes.

What Kubernetes gives you

- **Self-healing** — if a container or a whole node dies, Kubernetes replaces it automatically.
- **Automatic scaling** — the Horizontal Pod Autoscaler adds and removes Pods based on CPU or custom metrics.
- **Load balancing and service discovery** — a stable Service name and IP in front of a changing set of Pods.
- **Automated rollouts and rollbacks** — update gradually, and revert with one command if health checks fail.
- **Declarative configuration** — you describe the desired state in YAML; Kubernetes continuously makes reality match it.
- **Storage orchestration** — mount local, cloud or network storage into Pods.
- **Secret and config management** — inject configuration and credentials without rebuilding images.

THE SINGLE MOST IMPORTANT IDEA IN KUBERNETES

It is a DECLARATIVE, self-correcting system. You never say "start a container". You say "I want 3 replicas of this image running". A control loop then compares desired state with actual state, forever, and fixes any difference. Kill a Pod and a new one appears in seconds — you did not ask for that, the loop did it.

6.2 Kubernetes architecture

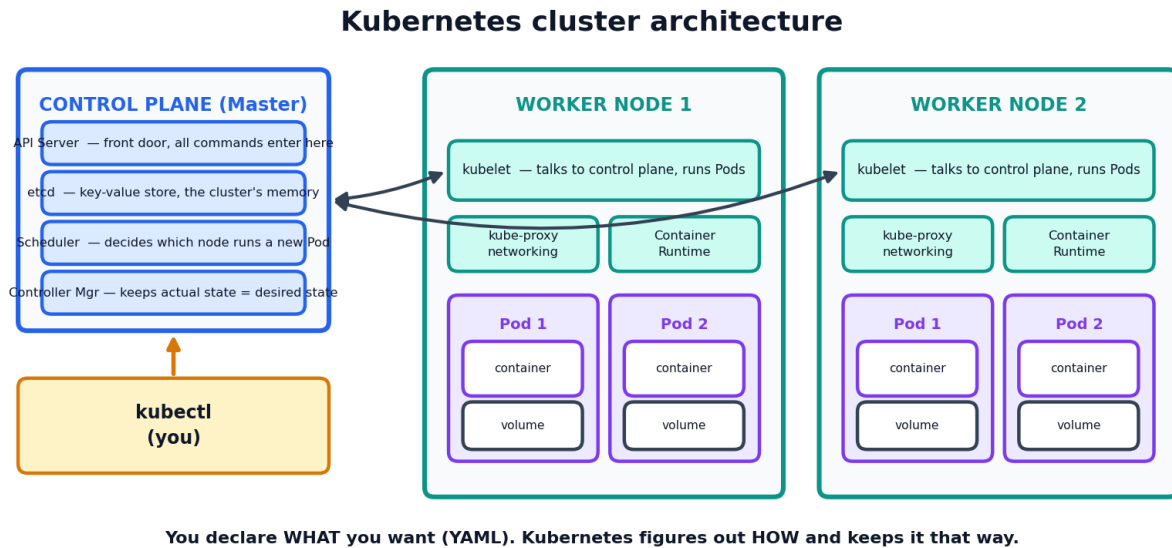


Figure 6.1 — Control Plane and Worker Nodes in a Kubernetes cluster

A cluster has two kinds of machine: the **Control Plane** (the brain that makes decisions) and the **Worker Nodes** (the muscle that runs your containers).

Control Plane components

Component	Responsibility
kube-apiserver	The front door of the cluster. Every command, from kubectl or from other components, goes through it. It validates requests and is the only component that talks to etcd.
etcd	A distributed key-value store holding the entire cluster state — the single source of truth. Back this up; lose it and you lose the cluster's memory.
kube-scheduler	Watches for newly created Pods with no node assigned and picks the best node for each, based on resource requests, affinity rules and constraints.
kube-controller-manager	Runs the control loops (Node controller, ReplicaSet controller, Job controller...) that constantly drive actual state toward desired state.
cloud-controller-manager	Talks to the cloud provider's API for load balancers, storage volumes and node lifecycle.

Worker Node components

Component	Responsibility
kubelet	The agent on every node. It receives Pod specifications from the API server, makes sure those containers are running and healthy, and reports status back.
kube-proxy	Maintains the network rules on the node so Services can reach the right Pods, and does simple load balancing between them.
Container Runtime	The software that actually runs containers — containerd or CRI-O (Docker's runtime was removed as a direct option in v1.24).

Walk-through: what happens when you deploy

- kubectl apply -f deployment.yaml** sends your YAML to the API server.

16. The API server validates it and writes the desired state into etcd.
17. The Deployment controller notices a Deployment with no ReplicaSet and creates one.
18. The ReplicaSet controller sees 0 Pods but 3 wanted, so it creates 3 Pod objects.
19. The scheduler sees 3 unassigned Pods and binds each to a suitable node.
20. The kubelet on each chosen node tells the container runtime to pull the image and start the containers.
21. kube-proxy updates network rules so the Service can route traffic to the new Pods.
22. If a Pod dies, the ReplicaSet controller notices $2 \neq 3$ and the cycle repeats. This is self-healing.

6.3 Pods and containers

A **Pod** is the smallest deployable unit in Kubernetes. You never deploy a container directly — you deploy a Pod, which wraps one or more containers that share a network namespace and storage volumes.

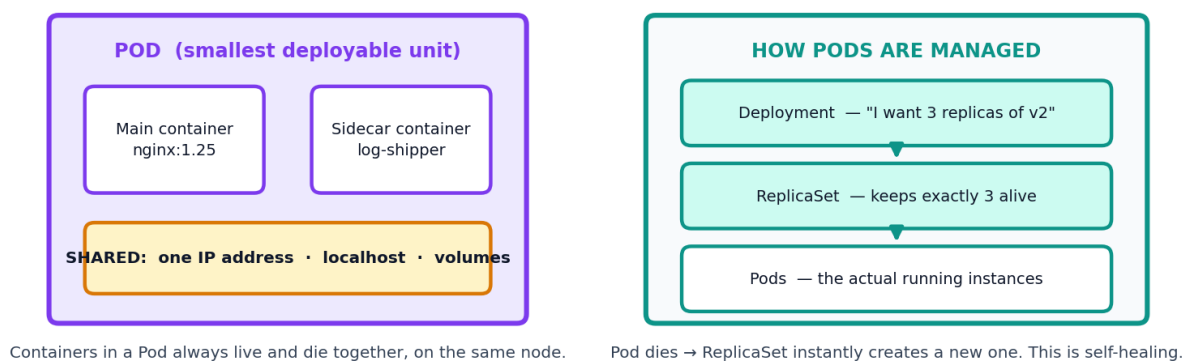


Figure 6.2 — Inside a Pod, and how Pods are managed by Deployments

What containers inside a Pod share

- **One IP address** — containers in the same Pod reach each other on **localhost**.
- **Storage volumes** — they can read and write the same mounted directories.
- **Lifecycle** — they are scheduled together onto one node, and are created and destroyed together.

Usually one container per Pod

The normal pattern is a single application container per Pod. A second container is added only when it is a helper that must live with the main one — this is called a sidecar. Classic sidecars: a log shipper, a metrics exporter, or a service-mesh proxy like Envoy.

Pods are disposable

Pods are mortal and are never repaired. If a Pod is unhealthy it is deleted and a brand-new one with a new IP is created. This is why you never talk to a Pod by IP — you go through a Service.

Core Kubernetes objects

Object	What it is for
Pod	One or more containers running together — the atom of deployment.
ReplicaSet	Keeps a stated number of identical Pods running at all times.
Deployment	Manages ReplicaSets to give you declarative updates, rolling upgrades and rollbacks. This is what you normally write.

Object	What it is for
Service	A stable name, IP and load balancer in front of a changing set of Pods (ClusterIP, NodePort, LoadBalancer).
Ingress	HTTP/HTTPS routing into the cluster by hostname and path, with TLS termination.
Namespace	A virtual partition of the cluster — separates dev, test and prod, or team from team.
ConfigMap	Non-secret configuration injected as environment variables or files.
Secret	Sensitive data (passwords, tokens, keys), base64-encoded and access-controlled.
DaemonSet	Runs exactly one copy of a Pod on every node — used for log and metric agents.
StatefulSet	For stateful apps that need stable network identities and persistent storage (databases).
Job / CronJob	Run-to-completion tasks; CronJob runs them on a schedule.
PersistentVolume / Claim	Storage that outlives Pods; a Claim is a Pod's request for it.

Example — a Deployment and a Service

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: portal-deployment
spec:
  replicas: 3                # DESIRED STATE: always keep 3 Pods
  selector:
    matchLabels:
      app: portal
  template:                  # the Pod blueprint
    metadata:
      labels:
        app: portal
    spec:
      containers:
        - name: portal
          image: student-portal:1.0
          ports:
            - containerPort: 3000
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
---
# service.yaml - a stable front door for those 3 Pods
apiVersion: v1
kind: Service
metadata:
  name: portal-service
spec:
  type: LoadBalancer
  selector:
```

```

    app: portal                # matches the Pod label above
  ports:
    - port: 80
      targetPort: 3000

```

Essential kubectl commands

```

kubectl apply -f deployment.yaml    # create or update from a file
kubectl get pods                   # list Pods
kubectl get pods -o wide           # ...with node and IP
kubectl get all                    # everything in the namespace
kubectl describe pod <name>        # detailed status + events (debugging #1)
kubectl logs <pod>                 # container logs
kubectl logs -f <pod>              # follow them
kubectl exec -it <pod> -- bash      # shell inside a Pod
kubectl scale deployment portal-deployment --replicas=5
kubectl set image deployment/portal-deployment portal=student-portal:2.0
kubectl rollout status deployment/portal-deployment
kubectl rollout undo deployment/portal-deployment # instant rollback
kubectl delete -f deployment.yaml
kubectl get nodes                   # cluster machines

```

6.4 Container orchestration basics

Container orchestration is the automated management of the full lifecycle of containers across a group of machines. Kubernetes is the dominant orchestrator; alternatives include Docker Swarm (simpler, less powerful), HashiCorp Nomad and cloud services such as AWS ECS.

Function	What the orchestrator does
Scheduling / placement	Chooses which node has enough CPU, memory and the right labels for each container.
Self-healing	Restarts failed containers, replaces Pods on dead nodes, kills ones failing health checks.
Scaling	Adds or removes replicas based on load (HPA), and even adds or removes nodes (Cluster Autoscaler).
Service discovery	Gives each service a stable DNS name; resolves it to healthy Pods.
Load balancing	Spreads incoming requests across all healthy replicas.
Rolling updates & rollback	Replaces old versions gradually; reverts automatically if health checks fail.
Configuration & secrets	Injects environment-specific config without rebuilding the image.
Storage orchestration	Attaches persistent volumes to the right Pods on the right nodes.

Two health checks worth naming: a **liveness probe** asks "is this container still alive?" (fail → restart it), and a **readiness probe** asks "is it ready for traffic?" (fail → stop sending it requests, but leave it running).

6.5 Monitoring and observability

Monitoring tells you **that** something is wrong — a dashboard shows CPU at 98%. **Observability** is the property of a system that lets you work out **why**, from the data it already emits, including for failures you never anticipated.

The three pillars of observability



Figure 6.3 — The three pillars of observability

6.6 Prometheus

Prometheus is an open-source monitoring and alerting system built at SoundCloud in 2012 and now a graduated CNCF project — the second project to graduate, after Kubernetes itself. It has become the de facto standard for monitoring containerised systems.

Defining characteristics

- **Pull-based** — Prometheus scrapes an HTTP `/metrics` endpoint on each target on a schedule. It is not pushed to.
- **Time-series database** — purpose-built for numeric data stamped with time.
- **Multi-dimensional data model** — every metric can carry labels, e.g. `http_requests_total{method="GET", status="500"}`.
- **PromQL** — a powerful query language for slicing, aggregating and alerting on that data.
- **Service discovery** — automatically finds new targets in Kubernetes, so new Pods are monitored without any config change.
- **Standalone and reliable** — each server is self-contained with local storage and no external dependencies, so it still works during an outage.

6.7 Time series data

Definition: A time series is a sequence of measurements of the same thing, recorded at successive points in time. Each data point is simply (timestamp, value), and the series is identified by a metric name plus a set of labels.

```
# One time series, identified by name + labels:
http_requests_total{job="api", method="GET", status="200", instance="10.1.2.3:8080"}

# Its data points:
1735689600 -> 15230
1735689615 -> 15267
1735689630 -> 15301

# Changing ANY label value creates a DIFFERENT time series.
```

CARDINALITY — THE CLASSIC MISTAKE

Every unique combination of labels is a separate time series. Adding a label like `user_id` or `request_id` creates millions of series and will bring Prometheus to its knees. Keep labels low-cardinality: `method`, `status`, `endpoint`, `region` — yes. `IDs`, `emails`, `timestamps` — never.

6.8 Metrics collection and metric types

The three metric types you must be able to tell apart

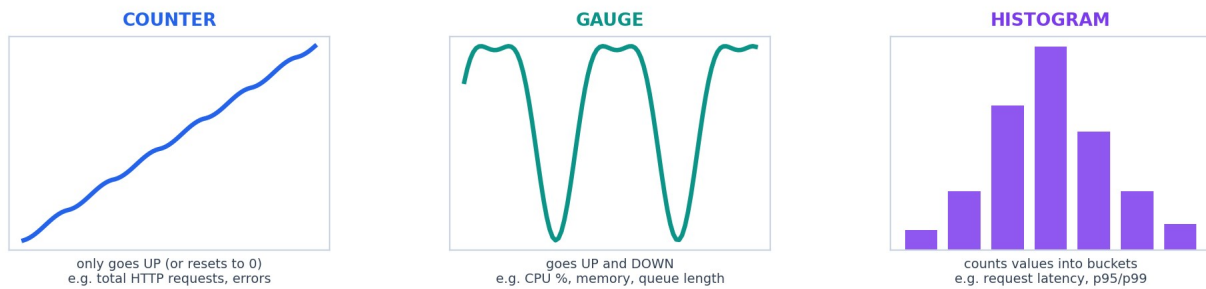


Figure 6.4 — Counter, Gauge and Histogram

Type	Behaviour	Use it for	Example metric
Counter	Only ever increases; resets to 0 on restart.	Things you count cumulatively.	http_requests_total, errors_total
Gauge	Goes up and down freely.	A current value you sample.	memory_usage_bytes, queue_length, temperature
Histogram	Counts observations into configurable buckets, and also tracks sum and count.	Distributions — latency, request size. Quantiles computed on the server.	http_request_duration_seconds
Summary	Like a histogram, but quantiles are calculated on the client.	Similar cases; cannot be aggregated across instances.	rpc_duration_seconds

COUNTER OR GAUGE?

Ask: can this number ever go DOWN? "Total requests served" cannot → Counter. "Requests currently in flight" can → Gauge. On a counter you almost always query `rate()`, not the raw value, because the raw number alone is meaningless.

A few PromQL queries

```

up                                # 1 if a target is reachable, 0 if not

# requests per second over the last 5 minutes:
rate(http_requests_total[5m])

# error rate as a percentage:
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m])) * 100

# 95th percentile latency:
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

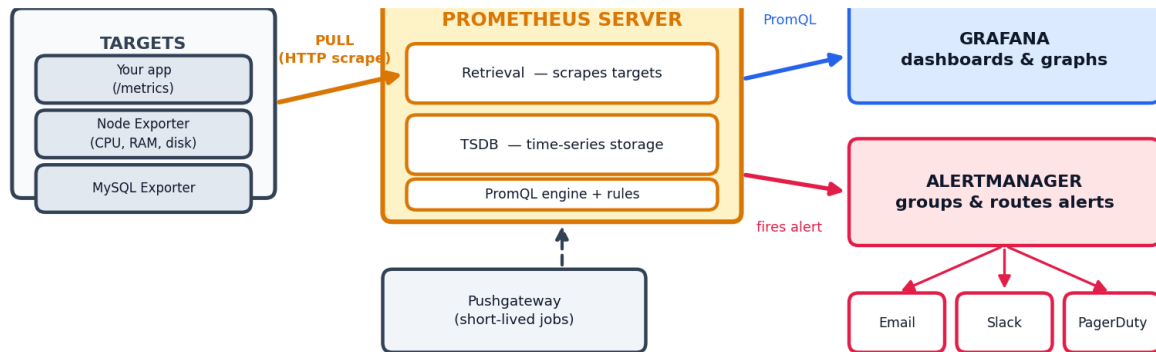
# memory usage per pod:
sum(container_memory_usage_bytes) by (pod)

# free disk below 15%:
node_filesystem_avail_bytes / node_filesystem_size_bytes < 0.15

```

6.9 Prometheus architecture

Prometheus + Grafana monitoring architecture



Prometheus PULLS metrics on a schedule. Grafana only draws; it stores nothing.

Figure 6.5 — Prometheus server, exporters, Alertmanager and Grafana

Component	Role
Prometheus Server	The core. Contains the retrieval (scraping) engine, the local time-series database (TSDB), and the PromQL query engine and rule evaluator.
Targets / instrumented apps	Applications that expose their own metrics on an HTTP /metrics endpoint using a client library (Java, Go, Python, Node).
Exporters	Small adapters that expose metrics for software that cannot be modified. Node Exporter (machine CPU/RAM/disk), MySQL Exporter, Blackbox Exporter (probes URLs), cAdvisor (containers).
Service Discovery	Finds targets automatically from Kubernetes, Consul, EC2 or a static file — so new Pods are scraped without editing config.
Pushgateway	For short-lived batch jobs that finish before they can be scraped; they push their result here and Prometheus scrapes the gateway.
Alertmanager	Receives fired alerts, then deduplicates, groups, silences and routes them to email, Slack, PagerDuty, or a webhook.
Grafana / UI	Queries Prometheus with PromQL and draws the dashboards. Prometheus has a basic built-in UI; Grafana is what teams actually use.

Minimal prometheus.yml

```

global:
  scrape_interval: 15s      # how often to scrape every target
  evaluation_interval: 15s  # how often to evaluate alert rules

alerting:
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']
  
```

```
rule_files:
  - 'alert_rules.yml'

scrape_configs:
  - job_name: 'prometheus'      # Prometheus monitors itself
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node'           # machine-level metrics
    static_configs:
      - targets: ['192.168.1.10:9100', '192.168.1.11:9100']

  - job_name: 'student-portal'
    metrics_path: /metrics
    static_configs:
      - targets: ['portal-service:3000']
```

6.10 Grafana and dashboard creation

Grafana is an open-source visualisation and analytics platform. It stores no metrics of its own — it queries data sources (Prometheus, Loki, Elasticsearch, MySQL, CloudWatch) and turns the results into dashboards.

Building a dashboard — the steps

- 23. Add the data source:** Configuration → Data sources → Prometheus → set the URL (e.g. `http://prometheus:9090`) → Save & test.
- 24. Create a dashboard:** + → New dashboard → Add visualization.
- 25. Write the query:** enter PromQL, e.g. `rate(http_requests_total[5m])`.
- 26. Choose the panel type:** Time series for trends, Stat for a single big number, Gauge for a value against a max, Bar chart, Table, Heatmap for histograms.
- 27. Format it:** set the unit (bytes, seconds, percent), a sensible axis range, and thresholds that turn the panel amber and red.
- 28. Add variables:** a `$namespace` or `$pod` dropdown makes one dashboard reusable across every service.
- 29. Arrange and save:** group related panels into rows, set the default time range and auto-refresh, then save. Export the JSON into Git so the dashboard is version-controlled too.

Useful extras: annotations mark deployments on the graph so you can see a spike start exactly when you released; and Grafana can define its own alert rules as well.

6.11 Logging and alerting

Logging

Logs are timestamped records of discrete events. In a distributed system you must centralise them — hunting through 40 machines is not viable.

Stack	Components
ELK / Elastic Stack	Elasticsearch (store & search) + Logstash (collect & transform) + Kibana (visualise). Filebeat ships the logs.
EFK	The same, with Fluentd or Fluent Bit instead of Logstash — lighter, common in Kubernetes.

Stack	Components
Grafana Loki	Indexes only labels, not the full text — much cheaper. Queried with LogQL and viewed in Grafana beside your metrics.

Good practice: use **structured logging** (JSON, not free text) so logs are machine-queryable; use consistent **log levels** (DEBUG, INFO, WARN, ERROR, FATAL); include a **correlation ID** so one request can be followed across services; and never log passwords, tokens or personal data.

Alerting

An alert is a rule that watches a metric and notifies a human when it crosses a threshold for a sustained period.

```
# alert_rules.yml
groups:
  - name: portal-alerts
    rules:
      - alert: HighErrorRate
        expr: sum(rate(http_requests_total{status=~"5.."}[5m]))
              / sum(rate(http_requests_total[5m])) > 0.05
        for: 5m # must stay true for 5 min - avoids flapping
        labels:
          severity: critical
        annotations:
          summary: "5xx error rate above 5%"
          description: "Error rate is {{ $value | humanizePercentage }}."

      - alert: InstanceDown
        expr: up == 0
        for: 2m
        labels: { severity: critical }
        annotations:
          summary: "{{ $labels.instance }} has been down for 2 minutes"
```

ALERT ON SYMPTOMS, NOT CAUSES

Alert on what users feel — high error rate, high latency, service unreachable. Do not page a human at 3 a.m. because CPU touched 80%; that is a dashboard metric, not an alert. Every alert should be urgent, actionable and link to a runbook. Alerts nobody acts on train the team to ignore all alerts.

6.12 Performance monitoring basics

Two well-known frameworks tell you what to watch.

Framework	What to measure	Best for
The Four Golden Signals (Google SRE)	Latency (how long requests take), Traffic (how much demand), Errors (rate of failed requests), Saturation (how full the system is).	User-facing services
USE Method	For every resource: Utilisation, Saturation, Errors.	Infrastructure — CPU, memory, disk, network
RED Method	Rate, Errors, Duration.	Microservices and APIs

What to monitor at each layer

Layer	Key metrics
Infrastructure	CPU %, memory used/available, disk space and I/O, network throughput, node availability.
Container / Kubernetes	Pod restarts, Pods pending, CPU/memory vs requests and limits, OOMKilled events, node pressure.
Application	Request rate, error rate, response time (p50/p95/p99), throughput, queue depth, thread pool usage.
Database	Query latency, active connections, slow queries, replication lag, cache hit ratio.
Business / user	Sign-ups per minute, checkout success rate, revenue per hour — the metrics that show real impact.

Also learn these terms: **SLI** (Service Level Indicator — the measurement, e.g. 99.2% of requests under 300 ms), **SLO** (Objective — the internal target, e.g. 99.5%), **SLA** (Agreement — the contractual promise to customers, with penalties), and **error budget** (the amount of failure your SLO allows — spend it on releasing features, and stop releasing when it runs out).

Appendix — Rapid Revision Sheet

A. One-line definitions

Term	Definition
DevOps	A culture and set of practices uniting Development and Operations to deliver software faster and more reliably.
CI	Every commit is automatically merged, built and tested.
Continuous Delivery	Every passing change is automatically made READY to release; a human approves.
Continuous Deployment	Every passing change is automatically RELEASED to production; no human.
IaC	Managing infrastructure through version-controlled configuration files instead of manual setup.
Idempotent	Running the same script once or many times produces the same result.
Git	A distributed version control system where every clone holds the complete history.
Jenkins	An open-source Java automation server used to build CI/CD pipelines.
Jenkinsfile	A file in the repository that defines the whole Jenkins pipeline as code.
Docker image	A read-only layered template containing an app and all its dependencies.
Container	A running instance of an image; shares the host OS kernel.
Kubernetes	An orchestration platform that automates deployment, scaling and management of containers.
Pod	The smallest deployable unit in Kubernetes — one or more containers sharing IP and storage.
Prometheus	A pull-based time-series monitoring and alerting system.
Grafana	A visualisation tool that queries data sources and renders dashboards.
Observability	The ability to understand a system's internal state from its metrics, logs and traces.

B. Command cheat sheet

Git

<code>git init</code>	<code>git clone <url></code>	<code>git status</code>
<code>git add .</code>	<code>git commit -m "msg"</code>	<code>git log --oneline --graph</code>
<code>git push origin main</code>	<code>git pull origin main</code>	<code>git fetch</code>
<code>git branch <name></code>	<code>git checkout -b <name></code>	<code>git switch <name></code>
<code>git merge <branch></code>	<code>git branch -d <name></code>	<code>git tag -a v1.0 -m "..."</code>
<code>git stash / git stash pop</code>	<code>git revert <hash></code>	<code>git reset --soft HEAD~1</code>

Docker

<code>docker build -t app:1.0 .</code>	<code>docker images</code>	<code>docker pull <image></code>
<code>docker run -d -p 8080:80 --name web <image></code>		<code>docker push user/app:1.0</code>
<code>docker ps / ps -a</code>	<code>docker logs -f <name></code>	<code>docker exec -it <name> bash</code>
<code>docker stop / start / rm</code>	<code>docker rmi <image></code>	<code>docker system prune -a</code>

```
docker volume create <v>    docker network create <n>    docker-compose up -d
```

Kubernetes

```
kubectl apply -f file.yaml    kubectl get pods|svc|all    kubectl get nodes
kubectl describe pod <p>    kubectl logs -f <p>    kubectl exec -it <p> -- bash
kubectl scale deployment <d> --replicas=5
kubectl set image deployment/<d> <c>=<image>:2.0
kubectl rollout status deployment/<d>    kubectl rollout undo deployment/<d>
kubectl delete -f file.yaml    kubectl get events --sort-
by=.metadata.creationTimestamp
```

C. Comparisons most likely to be asked

Question	The key distinction
Agile vs DevOps	Agile optimises development; DevOps extends that to delivery and operations.
CVCS vs DVCS	Central server holds the only full history vs every clone holds the full history.
Git vs GitHub	A tool on your machine vs a hosting website built around that tool.
Freestyle vs Pipeline	Configured by clicking in the UI vs defined as code in a Jenkinsfile.
Delivery vs Deployment	Ready to release (human approves) vs actually released (fully automatic).
VM vs Container	Virtualises hardware, has its own OS, GBs, minutes vs virtualises the OS, shares the kernel, MBs, seconds.
Image vs Container	A read-only template vs a running instance of it (class vs object).
Docker vs Kubernetes	Docker builds and runs one container; Kubernetes orchestrates thousands across many machines.
Counter vs Gauge	Only goes up vs goes up and down.
Monitoring vs Observability	Tells you THAT something is wrong vs lets you work out WHY.
Liveness vs Readiness probe	Alive? (fail → restart) vs ready for traffic? (fail → stop routing to it).
Blue-Green vs Canary	Switch 100% of traffic at once vs shift a small percentage gradually.

D. The whole pipeline in one paragraph

A developer writes code and commits it to a Git branch, then opens a Pull Request. A webhook triggers Jenkins, which checks the code out onto an agent, builds it with Maven, and runs unit and integration tests. If anything fails the team is told within minutes. If everything passes, Jenkins builds a Docker image, tags it with the build number, and pushes it to a registry. The pipeline deploys that exact image to staging and runs smoke tests. After approval — or automatically, under Continuous Deployment — Kubernetes performs a rolling update of the Deployment in production, replacing Pods a few at a time while readiness probes confirm each new Pod is healthy. Prometheus scrapes metrics from the new Pods, Grafana shows the dashboards, and Alertmanager pages someone if the error rate climbs. What those metrics reveal feeds back into the next sprint's planning — and the loop starts again.